Winding up Pointers

CSL-100

November 4, 2024

Example Function that Returns Pointer

char *strdup(const char *s);

- strdup creates a copy of the string (char array) passed as arguments
  - copy is created in dynamically allocated memory block of sufficient size
- returns a pointer to the copy created

char *strndup(const char *s, size_t num);

- Returns a pointer to a null-terminated byte string, which contains copies of at most num bytes from the string pointed to by s.

Returning Pointer: Beware

```
#include<stdio.h>
int *fun();
int main() {
  printf("%d", *fun());
}

int *fun() {
  int *p, i;
  p = &i;
  i = 10;
  return p;
}
```

OUTPUT



```
#include<stdio.h>
int *fun();
int main() {
  printf("%d", *fun());
}

int *fun() {
  int *p;
    p = (int*)malloc(sizeof(int));
  *p = 10;
  return p;
}
```

OUTPUT: 10

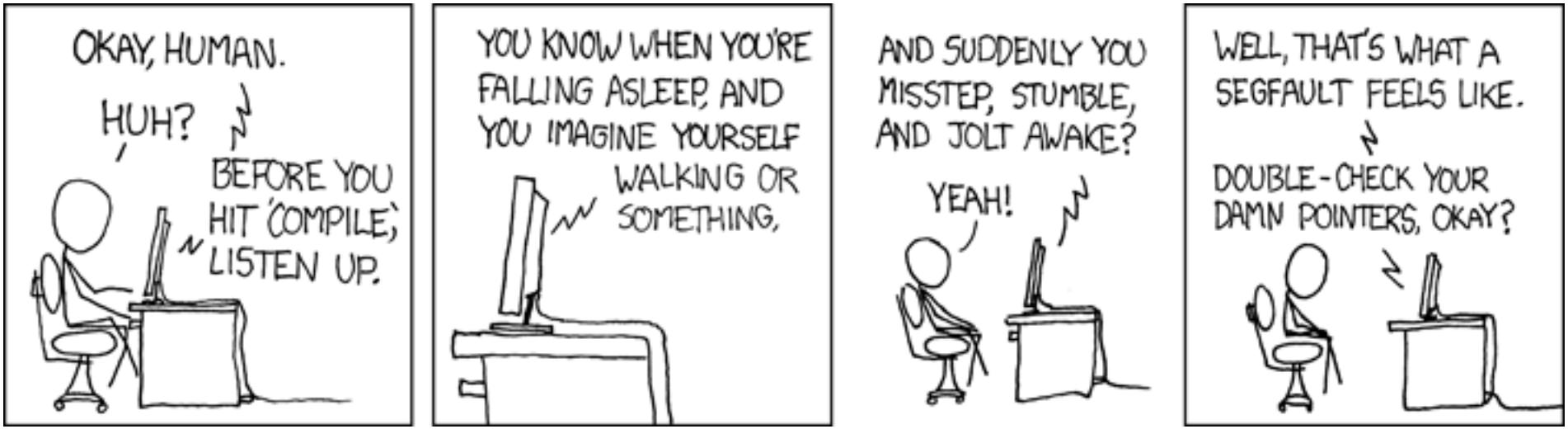
Returning Pointer: Beware

- The function stack (except for the return value) is gone once the function completes its execution.
  - All addresses of local variables and formal arguments become invalid
  - available for “reuse”
- But the heap memory, once allocated, remains until it is explicitly “freed”
  - even beyond the function that allocated it.
- addresses of static and global variables remain valid throughout the program.

Stack vs Heap: An Intuition

- Think of executing a function as writing on a classroom blackboard.
- Once the function finishes execution (the class is over), everything on the blackboard is erased.
- What if we want to retain a message, after class is over?
- Solution could be to post essential information on a “notice board”, which is globally accessible to all classrooms.
- The blackboard of a class is like the stack (possibly erased/overwritten in the next class), and the notice board is like the heap.

Common Issues and Errors



Source: http://www.xkcd.com/371

Common Issues and Errors

- Forgetting to malloc, forgetting to initialize allocated memory
- Not allocating enough space in malloc (e.g. Allocating 4 characters instead of 5 to store the string “IITK”.)
- Returning pointers to temporaries (called dangling pointers)
- Forgetting to free memory after use (called a memory leak.)
- Freeing the same memory more than once (runtime error), using free-d memory

Dangling Pointer

- Pointers that do not point to a valid object of the appropriate type
- Example: Returning address of a local variable from the function
- Another example:

```
{
  char *dp = NULL;
  /* ... */
  {
     char c;
     dp = &c;
  }
   /* c falls out of scope */
   /* dp is now a dangling pointer */
}
```

Memory Leaks

- Consider code:

```
1. int *a;
2. a = (int *)malloc(5*sizeof(int));
3. a = NULL;
```

- Memory is allocated to a at line 2.
- However, at line 3, a is reassigned NULL
- No way to refer to allocated memory!!
  - We can not even free it, as free-ing requires passing address of allocated block
- This memory is practically lost for the program (Leaked)
  - Ideally, memory should be freed before losing last reference to it

Multi-Dimensional Arrays

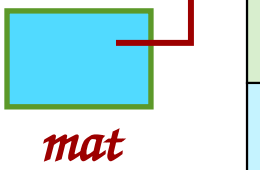
Declaration:

double mat[5][6];

int mat[5][6];

float mat[5][6];

mat is a 5 X 6 matrix of doubles (or ints or floats). It has 5 rows, each row has 6 columns, each entry is of type double.



2.1	1.0	-0.11	-0.87	31.5	11.4
-3.2	-2.5	1.678	4.5	0.001	1.89
7.889	3.333	0.667	1.1	1.0	-1.0
-4.56	-21.5	1.0e7	-1.0e-9	1.0e-15	-5.78
45.7	26.9	-0.001	1000.09	1.0e15	1.0

Multi-Dimensional Array vs. Multi-level pointer

```
int a[2][3];
```

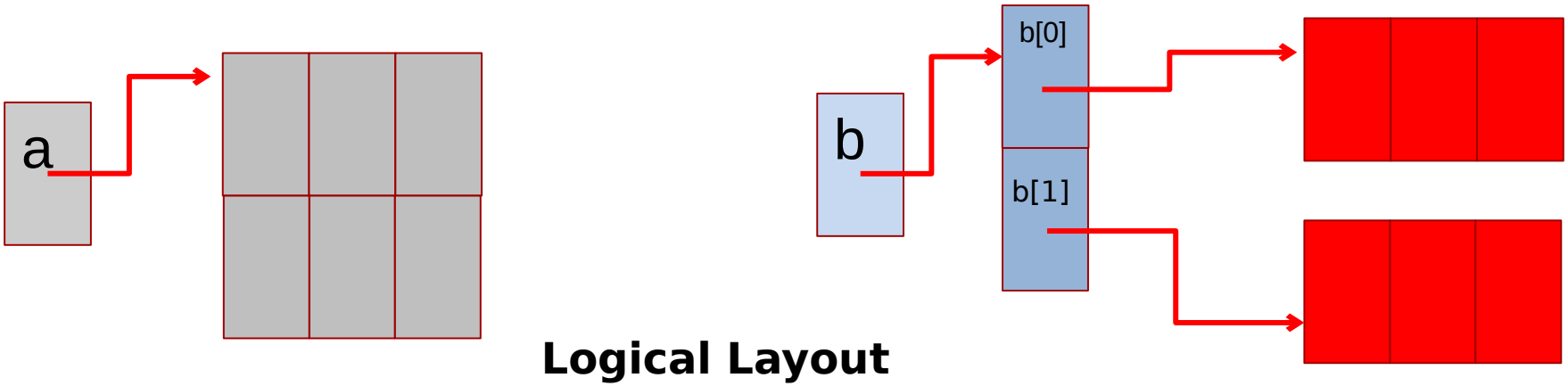
```
int **b;
b = (int**)malloc(2*sizeof(int*));
b[0] = (int*)malloc(3*sizeof(int));
b[1] = (int*)malloc(3*sizeof(int));
```

- Are these two equivalent?
  - Both a and b can hold 6 integer in a 2x3 grid like structure.
  - In case of a all 6 cells are consecutively allocated. For b, we have 2 blocks of 3 consecutive cells each.

Memory Layout

```
int a[2][3];
```

```
int **b;
b = (int**)malloc(2*sizeof(int*));
b[0] = (int*)malloc(3*sizeof(int));
b[1] = (int*)malloc(3*sizeof(int));
```



Logical Layout

Warning:

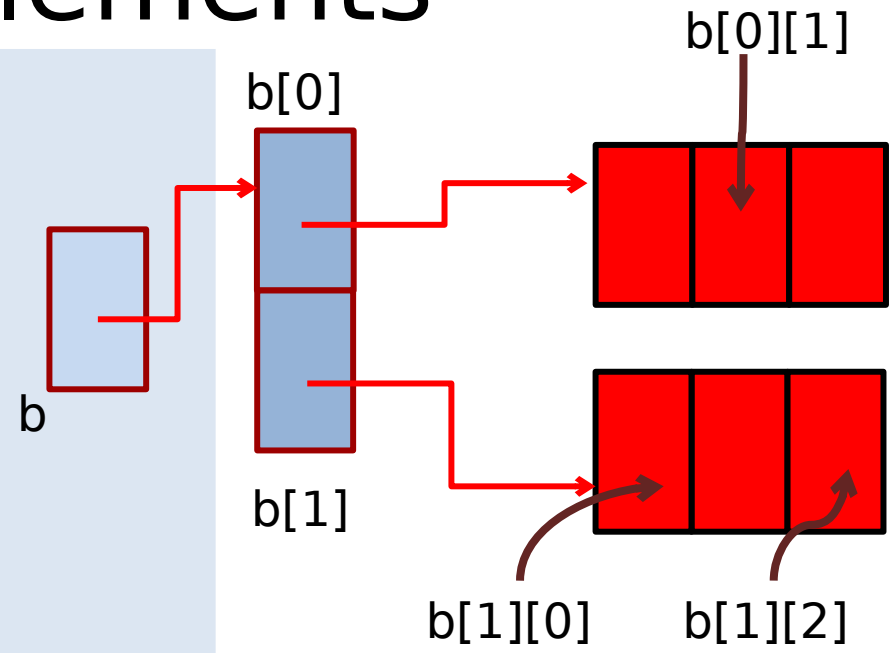
- **(*b+3) may not point to b[1][0].**
- **(*a+3) points to a[1][0].**

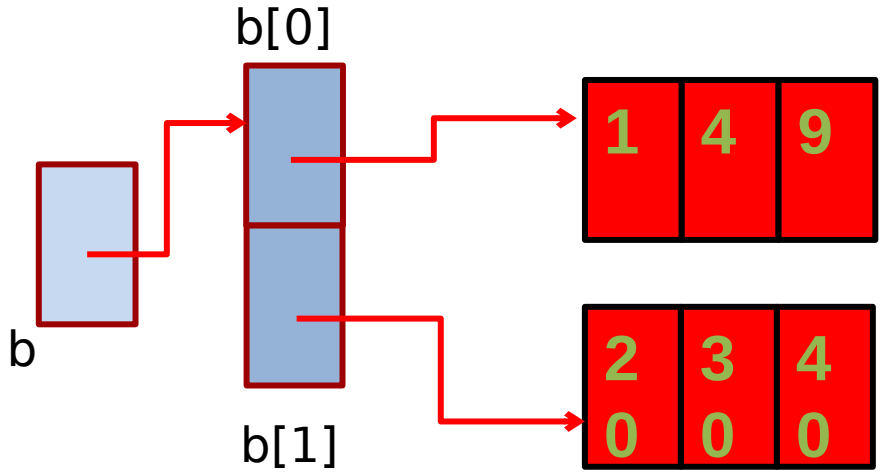
Indexing Elements

How to refer to an element of the array in the language of pointers?

- *b[0][1] is *(*b+1)*
- *b[1][0] is **(b+1)*
- *b[1][2] is *(*(b+1)+2)*

*In general, b[i][j] is *(*(b+i)+j)*





Expression	Value
**(b+1)	20
*(*b+1)	4
(**b+1)	2
*(*b+2)+2	11
((b+1)+2)+2	42

Pointers vs. Arrays: Indexing

- Matrix style notation A[i][j] is easier for humans to read
- Computers understand pointer style notation *(*(p + i) + j)
  - More efficient in some cases
- Be extremely careful with brackets
  - **(p + i +j) *(*(p + i) + j) *(*p + i + j)

```
int a[3][3], i, j, *b, *c;

for (i=0; i<3; i++)
    for (j=0; j<3; j++)
        a[i][j] = pow((i+3),(j+1));

b = *a;
c = *(a+2);
for (i=0; i<3; i++)
    printf("%d ", b[i]);
printf("\n");

for (i=0; i<3; i++)
    printf("%d ", *(c+i));
```

*At this point, array **a** is:*

3	9	27
4	16	64
5	25	125

*What do **b** and **c** point-to here?*

b is a pointer to a[0][0]?

c is a pointer to a[2][0]?

OUTPUT

3 9 27

5 25 125

```
int a[3][3], i, j, *b, *c;

for (i=0; i<3; i++)
    for (j=0; j<3; j++)
        a[i][j] = pow((i+3),(j+1));

b = *a;
c = *(a+2) + 1;
for (i=0; i<3; i++)
    printf("%d ", b[i]);
printf("\n");

for (i=0; i<2; i++)
    printf("%d ", *(c+i));
```

At this point, array **a** is:

3	9	27
4	16	64
5	25	125

What do **b** and **c** point-to here?

b is a pointer to a[0][0]?

c is a pointer to a[2][1]?

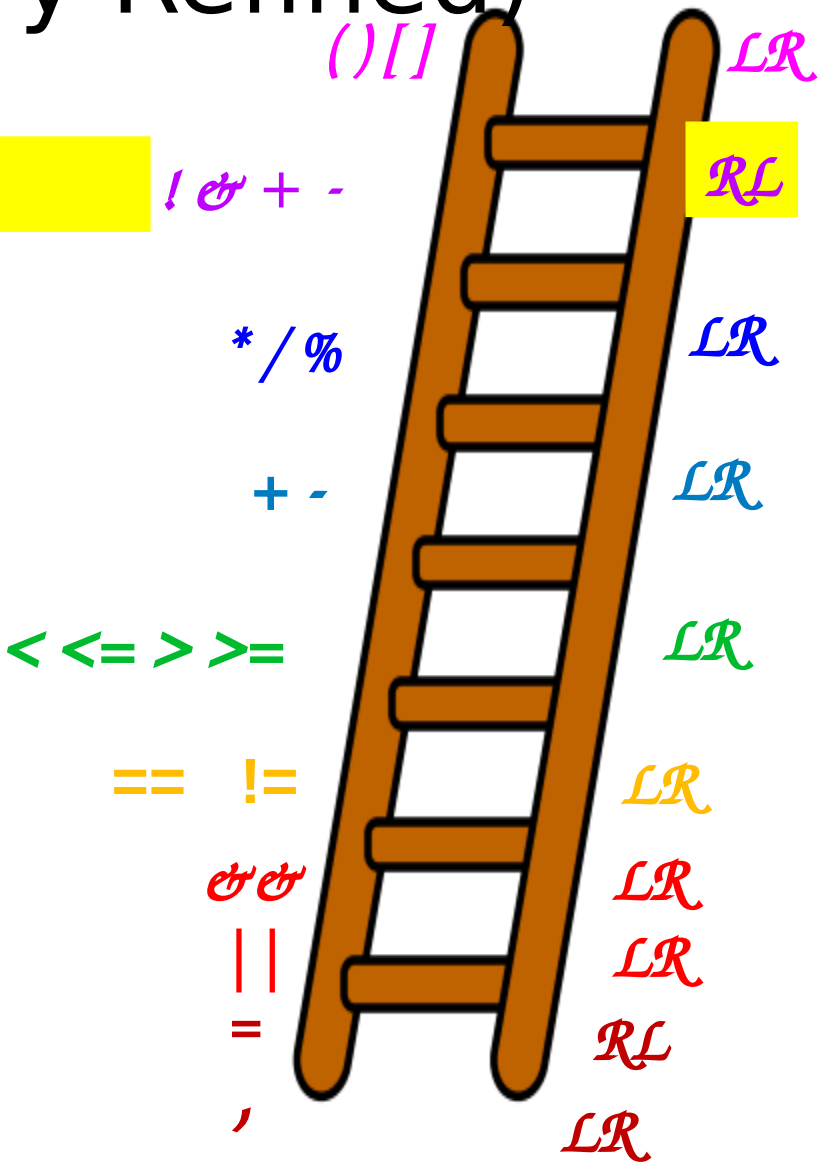
OUTPUT

3 9 27

25 125

note the change

Precedence (Unary Refined)



Array of Pointers

- Consider the following declaration

  int *arr[10];

- arr is a 10-sized array of pointers to integers
- How can we have equivalent dynamic array?

```
int **arr;
arr = (int **) malloc ( 10 * sizeof(int *) );
```

Array of Pointers

```
int **arr;
arr = (int**) malloc ( 10)* sizeof(int *)
```

- Note that individual elements in the array arr (arr[0], … arr[9]) are NOT allocated any space. Uninitialized.
- We need to do it (directly or indirectly) before using them.

```
int j;
for (j = 0; j < 10; j++)
    arr[j] = (int*) malloc (sizeof(int));
```

Pointers to an Array

```
int main()
{
   int arr[5] = { 1, 2, 3, 4, 5 };
   int *ptr = arr;
   prinf("%p\n", ptr);
   return 0;
}
```

- pointer *ptr* points to the 0th element of the array.
- Similarly, we can also declare a pointer that can point to whole array instead of only one element of the array.

Pointers to an Array (Contd.)

data_type (*var_name) [size_of_array];

int (*ptr)[10];

- Here *ptr* is pointer that can point to an array of 10 integers.
- Since subscript have higher precedence than indirection, it is necessary to enclose the indirection operator and pointer name inside parentheses.
- Here the type of ptr is "**pointer to an array of 10 integers**"

Pointers to an Array (Contd.)

```
int *p; // Pointer to an integer
int (*ptr)[5]; // Pointer to an array of 5 integers
int arr[5] = { 1, 2, 3, 4, 5 };
p = arr;  // Points to 0th element of the arr.
ptr = &arr; // Points to the whole array arr.
printf("p = %p, ptr = %p\n", p, ptr);

p++;
ptr++;
printf("p = %p, ptr = %p\n", p, ptr);
```

Result:

p = 0x7fff4f32fd50, ptr = 0x7fff4f32fd50
p = 0x7fff4f32fd54, ptr = 0x7fff4f32fd64

Example

- Array of pointers

```
int *ptr[10];
int a[10]={0,1,2,3,4,5,6,7,8,9};
 *ptr=a;
 printf("%d",*ptr[1]);
```

Output: Segfault

- Pointer to array

```
int (*ptr)[10];
int
a[10]={99,1,2,3,4,5,6,7,8,9};
ptr=&a;
printf("%d",(*ptr)[1]);
```

Output: 1

Class Quiz

- The following program illustrates the value of declarations of the form ***int (*ptr)[2]*** .

```
#include<stdio.h>
int main() {
        int a[] = {1,2,3};
        int (*ptr)[2] = &a;

        printf("%d\n", (*ptr)[0]);          printf("%d\n", (*ptr)
[1]);

        (*ptr)[0] = -1;
        printf("%d\n", a[0]);
        return 0;
}
```

An equivalent assignment is:

```
int (*ptr)[2];
ptr = &a;
```

OUTPUT:

```
1
2
-1
```

Next Class

- Introduction to Structure